

1. Introduction & V15 Engine Architecture

Executing massive computational graphs on hardware with strict on-chip fast memory presents a combinatorial optimization challenge. The latency is entirely governed by a roofline model that heavily penalizes repeated data spillage to slow memory. Our submission, the **V15 Engine**, minimizes end-to-end execution cycles through **Adaptive Dynamic Programming (DP)**, fine-grained **Split-K Pipelining**, strict memory limits enforced via **Deduplicated Retention Set** heuristics, and **Two-Level OpenMP Parallelism** that accelerates the search across all available cores.

The engine handles both trivial (5 nodes) and massive (105+ nodes) DAGs without violating the 10-minute timeout. The binary is compiled statically with `-fopenmp` and deployed inside a minimal Docker container based on Ubuntu 22.04. V15 achieves **4.12M total cycles** across all 5 public benchmarks in **8.75 seconds** wall-clock on 8 cores — a 17.9x speedup over V14.

2. Algorithmic Foundation: Why DP?

The core challenge is grouping dependent operations sequentially without saturating the capacity constraint C_{fast} . Greedy algorithms fall into local optima — aggressively retaining intermediate tensors often leads to OOM crashes deeper in the graph. RL approaches require exorbitant training times and struggle with strict generalization guarantees.

We define the problem via linear **Dynamic Programming** over a topologically sorted node sequence. The DP state $dp[i]$ represents the minimal global latency required to execute the graph up to node i : $dp[i] = \min_{j < i} (dp[j] + L(S_j \rightarrow i, M_{retention}))$ where L calculates the analytical roofline latency of the candidate subgraph, parameterized by the retained tensors from the preceding step.

2.1 Adaptive Window Sizing

Exhaustive evaluation of all $j \in [0, i-1]$ is $O(N^2)$, intractable for dense $N \geq 60$ topologies. V15 uses **Adaptive Windowing** with significantly wider windows than V14, enabled by the performance improvements described in Section 5:

N	W	R (modes)
≤ 32	N (full)	10
≤ 40	16	10
≤ 65	32	10
≤ 105	20	10
> 105	8	4

The wider windows in V15 allow the DP to find better partitions — benchmark B13 improved from 424K to 378K cycles (−11%) solely from finding a superior subgraph grouping.

3. Generating Computational Granularity

Once a candidate subgraph S is identified, determining its internal execution tile dimensions $[w, h, k]$ is the physical optimization lever. V15 mathematically curates candidates from: (1) multiples/divisors of native hardware granularity, (2) exact dimensional bounds of the subgraph's topological outputs, and (3) power-of-2 factors.

3.1 Split-K Pipelining

MatMul operations require holding LHS , RHS , and $Output$ simultaneously. V15 evaluates the inner reduction depth k via **Split-K Pipelining**: using small k values, the system accumulates partial dot products within a pinned $Output$ tensor in fast memory, iterating over the reduction dimension K sequentially. This drastically slashes the working set requirement and bypasses OOM crashes in large GEMMs.

4. Fast Memory Retention Strategies

Identifying which tensors $t \in T$ to persist in C_{fast} is NP-Hard. In our DP transition loop, $M_{retention}$ is generated by simulating 10 concurrent filtering modes:

- Mode 1:** Baseline eviction (spilling). Safe but bandwidth-bound.
- Modes 0, 3, 4, 5: Size-Based Packing.** Tensors sorted by physical size and greedily packed into C_{fast} or fractional thresholds (e.g., 75% capacity).
- Modes 2, 6, 9: Next-Use Proximity.** Tensors needed in the immediate horizon (within 6 operations) take priority.

4.1 Strict Hash Deduplication

Generating up to 10 retention sets creates overlapping configurations. V15 resolves duplicates via deterministic 64-bit FNV-1a hashing: $h = 0 \oplus \text{cbf29ce484222325} \oplus h \oplus t \times 0 \oplus \text{100000001b3}$. Only uniquely mapped retention subsets proceed to the heavy evaluation function.

5. V15 Concurrency Architecture

V15 fundamentally rearchitects the parallelism model compared to V14, fixing a critical nested OpenMP bug and introducing cache-optimized data structures.

5.1 Two-Level OpenMP Parallelism

V14 had a bug where `omp_set_max_active_levels(1)` prevented the inner granularity search from running in parallel. V15 fixes this with a clean two-level architecture:

Level 1 — Retention Mode Parallelism. Each DP transition evaluates up to R retention modes concurrently via `#pragma omp parallel for`. Every thread builds its own retention set independently, evaluates it through `find_best()`, and stores results in a thread-local struct. A sequential reduction picks the global minimum.

Level 2 — Granularity Search Parallelism. Inside `find_best()`, all (w, h, k) granularity combinations are flattened into a vector and distributed with `#pragma omp for schedule(dynamic, 4)`. Each thread maintains a local `Best` struct and merges via `#pragma omp critical`.

V15 properly enables `omp_set_max_active_levels(2)`, achieving true nested parallelism. This yields a measured **17.9x wall-clock speedup** over V14 (54.58s → 8.75s on 8 cores) while producing **bit-identical cycle counts**.

5.2 Cache-Optimized Hot Path

V15 replaces `std::unordered_set<size_t>` with a custom **flat bitset** (`TensorSet`) for all tensor membership operations. The bitset uses 64-bit words with $O(1)$ insert/lookup, dramatically reducing cache misses in the innermost evaluation loop which is called millions of times per benchmark.

Additionally, V15 **precomputes** a `SubgraphTensorInfo` struct once per DP transition — identifying produced/consumed/ephemeral tensors, LHS/RHS roles, and base costs — and shares this across all (w, h, k) granularity evaluations, eliminating redundant recomputation. An `OpType` enum replaces runtime string comparisons ("MatMul" vs "Pointwise").

6. Latency Evaluation Model

The core evaluation simulates hardware behavior analytically. For every candidate, latency is bounded by the roofline formula: $L = \max(T_{\text{Compute}}, T_{\text{Stream In}} + T_{\text{Stream Out}})$

6.1 Traversal Sequence Selection

When an execution profile divides the feature map into multiple tiles, the execution order modulates memory reuse. V15 simulates **Raster** (top-left to bottom-right) against **Snake** (alternating sweep) traversals. Snake traversals reuse intersecting boundaries of adjacent chunks in `C_fast`, and V15 injects snake ordering whenever it mathematically reduces stream bandwidth.

7. Results & Conclusion

Benchmark	Cycles	Time	Timeout
B1	87,840	0.005s	2s
B5	122,336	0.028s	5s
B9	2,768,241	0.995s	15s
B13	377,815	7.099s	30s
B17	767,764	0.618s	60s
Total	4,123,996	8.75s	—

The V15 engine merges DP with hyper-localized heuristics for chunking and pinning. The combination of fixed nested OpenMP parallelism, bitset-optimized hot paths, and precomputed tensor metadata delivers a 17.9x speedup over V14 while maintaining deterministic, optimal latency schedules. Total wall-clock across all benchmarks is under 9 seconds on 8 cores.